DevOps Hands-On Project 4: Convert the Kubernetes Lab into a Helm Chart

Company: SwiftCart Logistics **Level:** Beginner to intermediate DevOps **Duration:** 1 hours 30 mins **Prerequisites:** Docker, Docker Compose, Kubernetes/Minikube lab **Goal:** Package the existing Flask + PostgreSQL Kubernetes application as a reusable Helm chart.

1. Case study and problem

SwiftCart's application already runs on Kubernetes, but each environment requires several YAML manifests. The DevOps team wants repeatable installation, configurable replicas and settings, versioned packaging, upgrades and rollback. Students will convert the Kubernetes resources into a Helm chart.

2. What Helm adds

- **Chart:** package containing metadata, values and templates.
- **Chart.yaml:** chart name/version and app version metadata.
- **values.yaml:** configurable defaults.
- **Templates:** Kubernetes manifests parameterized with Helm expressions.
- **Release:** an installed instance of a chart.
- **Upgrade/rollback:** update a release and return to a previous revision.

Helm manages Kubernetes resources; it does not replace Kubernetes.

3. Architecture



The chart creates the app Deployment/Service, ConfigMap, Secret, PostgreSQL Deployment/Service, PVC and optional Ingress. It includes health probes and resource requests/limits.

4. Project files

```
swiftcart-helm-chart-project/
├── app/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── swiftcart/
│   ├── Chart.yaml
│   ├── values.yaml
│   ├── .helmignore
│   └── templates/
│       ├── _helpers.tpl
│       ├── configmap.yaml
│       ├── secret.yaml
│       ├── postgres-pvc.yaml
│       ├── postgres-deployment.yaml
│       ├── postgres-service.yaml
│       ├── deployment.yaml
│       ├── service.yaml
│       ├── ingress.yaml
│       └── NOTES.txt
└── values-lab.yaml
```

5. Install and verify prerequisites

Install Docker, kubectl, Minikube and Helm, then run:

```
docker --version
kubectl version --client
minikube version
helm version
```

References:

- Helm install: https://helm.sh/docs/intro/install/
- Minikube start: https://minikube.sigs.k8s.io/docs/start/
- kubectl tools: https://kubernetes.io/docs/tasks/tools/

6. Start Minikube and enable Ingress

```
minikube start --driver=docker
minikube status
kubectl get nodes
minikube addons enable ingress
kubectl get pods -n ingress-nginx
```

Wait for the controller:

```
kubectl wait --namespace ingress-nginx --for=condition=Ready pod --all --
timeout=180s
```

7. Build the application image

Run from the project root:

```
eval $(minikube docker-env)
cd app
docker build -t swiftcart-api:4.0.0 . or minikube image build -t swiftcart-
api:4.0.0 .
docker images | grep swiftcart-api
```

The chart defaults to this local image. If the cluster cannot see this Docker daemon, use `minikube image load` or a registry, and update the chart's image repository/tag values.

8. Explore the chart

```
cat swiftcart/Chart.yaml
cat swiftcart/values.yaml
find swiftcart -maxdepth 3 -type f
```

Key values include `replicaCount`, image repository/tag, app/service ports, Ingress host, PostgreSQL image, storage size, resource requests and limits.

The chart version (`version`) and application version (`appVersion`) are different: a chart-template change can increase the chart version without changing the app binary.

9. Lint and render before installing

```
helm lint ./swiftcart
helm template swiftcart-dev ./swiftcart --namespace swiftcart --debug
helm template swiftcart-dev ./swiftcart --namespace swiftcart -f values-lab.yaml
```

Review generated resource names, image tag, replica count, ConfigMap/Secret references, database Service hostname, PVC size and Ingress backend. `helm template` renders locally; it does not prove that the cluster can run every resource.

10. Install the chart

```
helm install swiftcart-dev ./swiftcart --namespace swiftcart --create-namespace --
wait --timeout 5m
```

Inspect the release and resources:

```
helm list -n swiftcart
helm status swiftcart-dev -n swiftcart
kubectl get all -n swiftcart
kubectl get configmap,secret,pvc,ingress -n swiftcart
helm get manifest swiftcart-dev -n swiftcart
```

The generated names are based on the release name plus chart name. For example, the app Service/Deployment is `swiftcart-dev-swiftcart`; PostgreSQL resources use `swiftcart-dev-swiftcart-postgres`.

11. Verify the app and database

```
kubectl rollout status deployment/swiftcart-dev-swiftcart -n swiftcart
kubectl get pods -n swiftcart
kubectl logs deployment/swiftcart-dev-swiftcart-postgres -n swiftcart
kubectl get ingress -n swiftcart
minikube ip
```

Add the actual Minikube IP to the hosts file as `swiftcart.local`. Linux/macOS: `/etc/hosts`. Windows: `C:\Windows\System32\drivers\etc\hosts`.

```
curl http://swiftcart.local/health
curl http://swiftcart.local/api/db-check
curl http://swiftcart.local/api/status
```

If Ingress access does not work with your driver, test the app using a port-forward:

```
kubectl port-forward service/swiftcart-dev-swiftcart 8080:80 -n swiftcart
```

Then visit `http://localhost:8080`. Port-forwarding is a fallback test, not a replacement for the Ingress exercise.

12. Understand configuration and secrets

```
kubectl describe configmap swiftcart-dev-swiftcart-config -n swiftcart
kubectl describe secret swiftcart-dev-swiftcart-db -n swiftcart
```

```
kubectl get pvc -n swiftcart
```

ConfigMap contains non-sensitive settings. Secret contains the DB username/password. The sample `swiftcartpass` is for classroom use only. Kubernetes Secret objects are not a substitute for access control, encryption at rest or an external secret manager. Never commit real credentials to source control.

13. Scale with Helm values

```
helm upgrade swiftcart-dev ./swiftcart --namespace swiftcart --set replicaCount=3
--wait
kubectl get deployment swiftcart-dev-swiftcart -n swiftcart
kubectl get pods -n swiftcart
helm get values swiftcart-dev -n swiftcart
```

Scale back:

```
helm upgrade swiftcart-dev ./swiftcart --namespace swiftcart --set replicaCount=2
--wait
```

The Deployment template reads `{{ .Values.replicaCount }}`, so changing a value changes the desired replica count without editing the template.

14. Use environment-specific values

```
helm upgrade swiftcart-dev ./swiftcart --namespace swiftcart -f values-lab.yaml --
wait
helm get values swiftcart-dev -n swiftcart
helm get values swiftcart-dev -n swiftcart --all
```

The sample override sets three replicas and DEBUG as the app log-level configuration. In real projects, keep environment overrides separate from chart templates, and manage production secrets securely.

15. Upgrade the application image

Build a new image tag:

```
eval $(minikube docker-env)
cd app
docker build -t swiftcart-api:4.0.1 . or minikube image build -t swiftcart-
api:4.0.1 .
```

Upgrade:

```
helm upgrade swiftcart-dev ./swiftcart --namespace swiftcart --set image.tag=4.0.1
--set app.version=4.0.1 --wait
kubectl rollout status deployment/swiftcart-dev-swiftcart -n swiftcart
helm history swiftcart-dev -n swiftcart
curl http://swiftcart.local/health
```

For a meaningful class exercise, students can change the app banner/version before building the new tag.

16. History and rollback

```
helm history swiftcart-dev -n swiftcart
helm rollback swiftcart-dev 1 -n swiftcart --wait
helm status swiftcart-dev -n swiftcart
```

Replace revision `1` with the desired revision shown by `helm history`. Rollback restores an earlier release revision, but it does not automatically reverse irreversible database schema changes or external side effects.

17. Package the chart

```
helm lint ./swiftcart
helm package ./swiftcart
helm show chart ./swiftcart
helm show values ./swiftcart
```

This produces an archive such as `swiftcart-0.1.0.tgz`. Optionally install it as a second release, but note that it creates another app/database stack and consumes additional local resources.

18. Troubleshooting

ImagePullBackOff

```
kubectl describe pod <POD-NAME> -n swiftcart
eval $(minikube docker-env)
docker images | grep swiftcart-api
```

PVC Pending

```
kubectl get pvc -n swiftcart
kubectl describe pvc swiftcart-dev-swiftcart-postgres -n swiftcart
kubectl get storageclass
```

Application cannot connect to DB

```
kubectl get pods,svc -n swiftcart
kubectl logs deployment/swiftcart-dev-swiftcart -n swiftcart
kubectl logs deployment/swiftcart-dev-swiftcart-postgres -n swiftcart
helm get manifest swiftcart-dev -n swiftcart
```

Ingress not reachable

```
minikube addons enable ingress
kubectl get pods -n ingress-nginx
kubectl describe ingress swiftcart-dev-swiftcart -n swiftcart
minikube ip
```

Helm install/upgrade issue

```
helm status swiftcart-dev -n swiftcart
helm history swiftcart-dev -n swiftcart
kubectl get events -n swiftcart --sort-by=.lastTimestamp
```

19. Assessment tasks

1. Explain the difference between raw Kubernetes YAML and a Helm chart.
2. Scale the application to four replicas using `--set`.
3. Create `values-student.yaml` with a different replica count and log level.
4. Build and deploy a new image tag.
5. Inspect Helm history and roll back to a previous revision.
6. Explain why real passwords must not be committed in `values.yaml`.
7. Explain how two teams can install the same chart with different release names, namespaces and values.

20. Final checklist

- [ ] `helm lint ./swiftcart` succeeds.
- [ ] `helm template` output has been inspected.
- [ ] Release installed successfully.
- [ ] App Deployment has desired replicas Ready.
- [ ] PostgreSQL Pod is Ready.
- [ ] ConfigMap, Secret and PVC exist.
- [ ] Ingress exists.
- [ ] `/health`, `/api/db-check` and `/api/status` work.
- [ ] Replica count changed via Helm values.
- [ ] Upgrade and rollback demonstrated.
- [ ] Chart packaged successfully.

21. Cleanup

```
helm uninstall swiftcart-dev -n swiftcart
kubectl delete namespace swiftcart
minikube stop
```

Storage cleanup depends on the StorageClass/PV reclaim policy. Back up data you need and inspect the PVC/PV before cleanup. Do not assume that uninstalling a release safely deletes or preserves database data in every cluster.

22. Lessons learned

1. Kubernetes manifests define desired resources.
2. Helm templates make resources configurable and reusable.
3. `values.yaml` separates configuration from templates.
4. Chart version and application version are distinct.
5. A release is an installed instance of a chart.
6. `helm lint` and `helm template` help catch problems before installation.
7. `helm upgrade` updates a release and `helm rollback` returns to an earlier revision.
8. Helm packages and manages Kubernetes resources; it does not replace Kubernetes.

23. References

Official Helm

- Documentation: https://helm.sh/docs/
- Installation: https://helm.sh/docs/intro/install/
- Charts: https://helm.sh/docs/topics/charts/
- Template guide: https://helm.sh/docs/chart_template_guide/
- Values files: https://helm.sh/docs/chart_template_guide/values_files/
- Command reference: https://helm.sh/docs/helm/
- Upgrade: https://helm.sh/docs/helm/helm_upgrade/
- Rollback: https://helm.sh/docs/helm/helm_rollback/
- Package: https://helm.sh/docs/helm/helm_package/

Kubernetes

- Deployments: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Services: https://kubernetes.io/docs/concepts/services-networking/service/
- ConfigMaps: https://kubernetes.io/docs/concepts/configuration/configmap/
- Secrets: https://kubernetes.io/docs/concepts/configuration/secret/
- Persistent Volumes: https://kubernetes.io/docs/concepts/storage/persistent-volumes/
- Ingress: https://kubernetes.io/docs/concepts/services-networking/ingress/

Minikube

- Start: https://minikube.sigs.k8s.io/docs/start/
- Addons: https://minikube.sigs.k8s.io/docs/handbook/addons/
- Ingress tutorial: https://minikube.sigs.k8s.io/docs/tutorials/ingress/

Instructor note

This is a training chart, not a production-ready PostgreSQL deployment. For production, consider a managed database or a carefully designed StatefulSet/operator, external secret management, backups, TLS, network policies, monitoring, image scanning, resource sizing and tested recovery procedures.

Learning progression: Docker → Docker Compose → Kubernetes/Minikube → Helm → Helm + CI/CD → monitoring → managed Kubernetes.